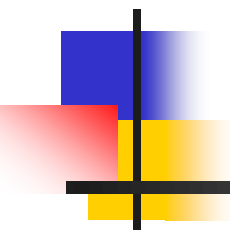
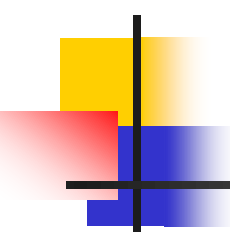


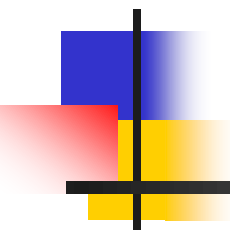
PART 2



RELATIONAL DATABASES



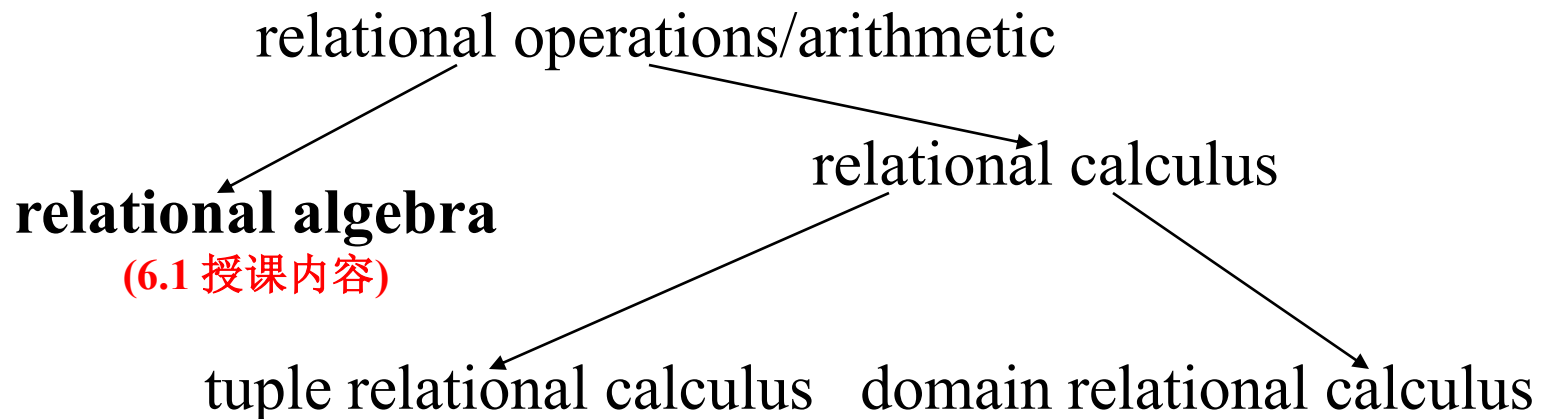
Chapter 6



Formal Relational Query Language

Main Parts in Chapter 6

- Categories of relational operations



- tuple relational calculus (元组关系演算, § 6.2)
- domain relational calculus (域关系演算, § 6.3)

Main Parts in Chapter 6

■ *Relational algebra*

- fundamental operations (§ 6.1.1-2)
- additional operations (§ 6.1.3)
- extended operations (§ 6.1.4)

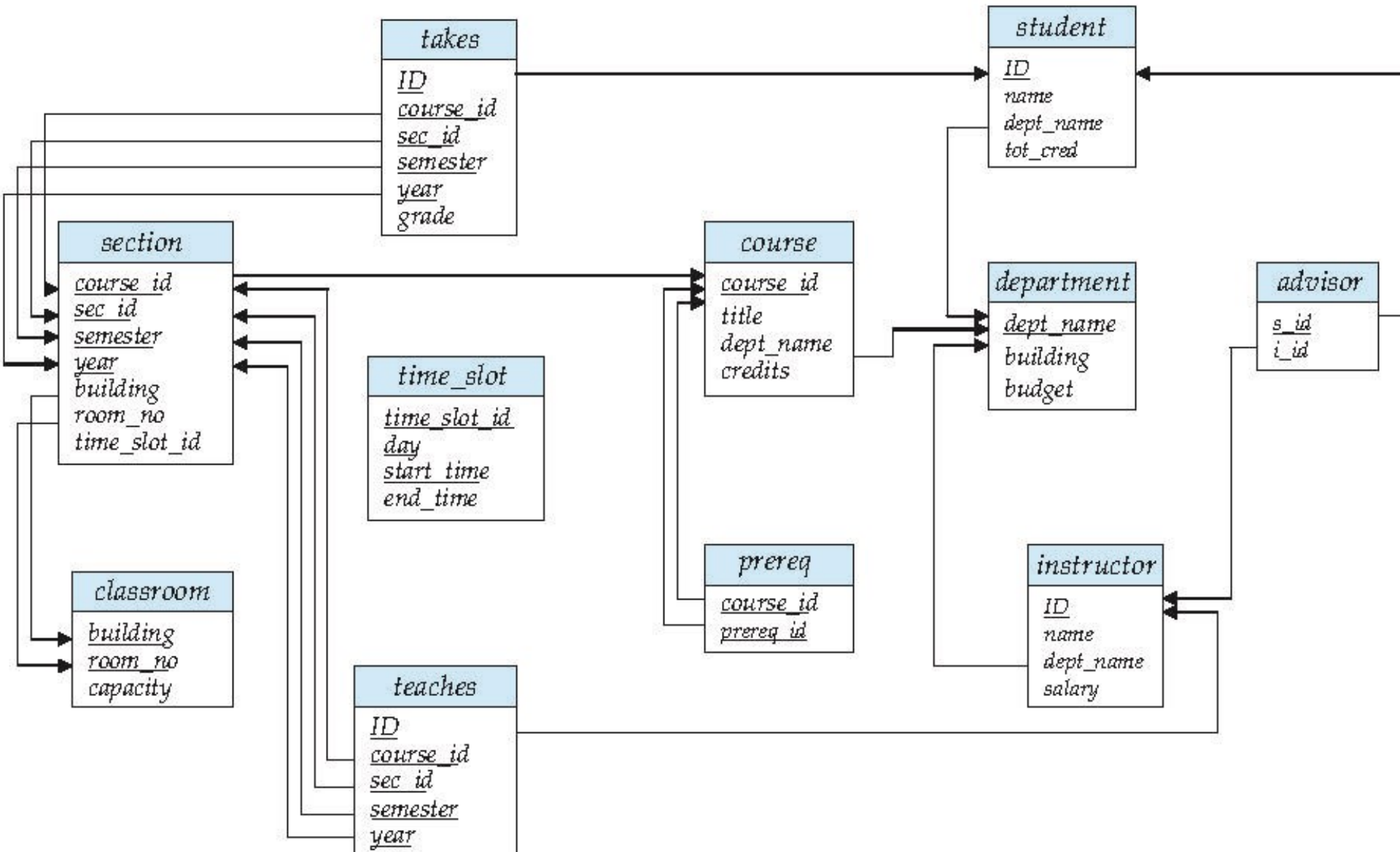


Fig. Schema Diagram

用处？！

- SQL语言的理论基础
- 实际DBMS环境下，观察SQL语句的查询执行计划——表示为关系代数操作，分析、判断、优化SQL语句
 - 提高SQL语句执行效率，保证数据库系统性能
— 数据库系统维护优化
- E.g. 观察select语句的查询执行计划不同，比较同一查询的不同实现方式

```

select distinct course_id, semester
from section
where semester = 'Fall' and year= 2009 and course_id in (select course_id
                                                           from section
                                                           where semester = 'Spring' and year= 2010)

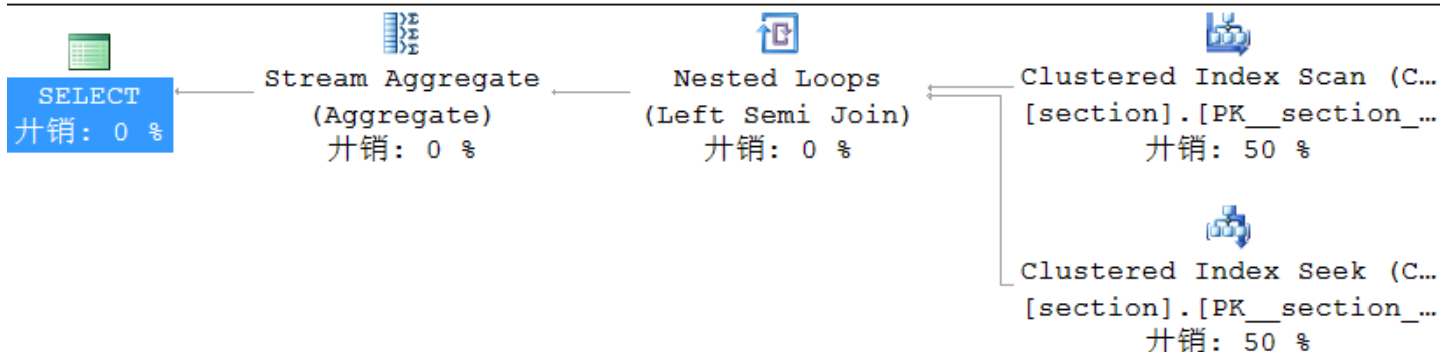
```

100 %

消息 执行计划

查询 1: (与该批有关的) 查询开销: 100%

select distinct course_id, semester from section where semester = 'Fall' and year= 2009



查询1: cost=87

```
(select distinct salary from instructor) |  
except  
(select distinct T.salary from instructor as T, instructor as S where T.salary < S.salary)
```

查询2: cost=13

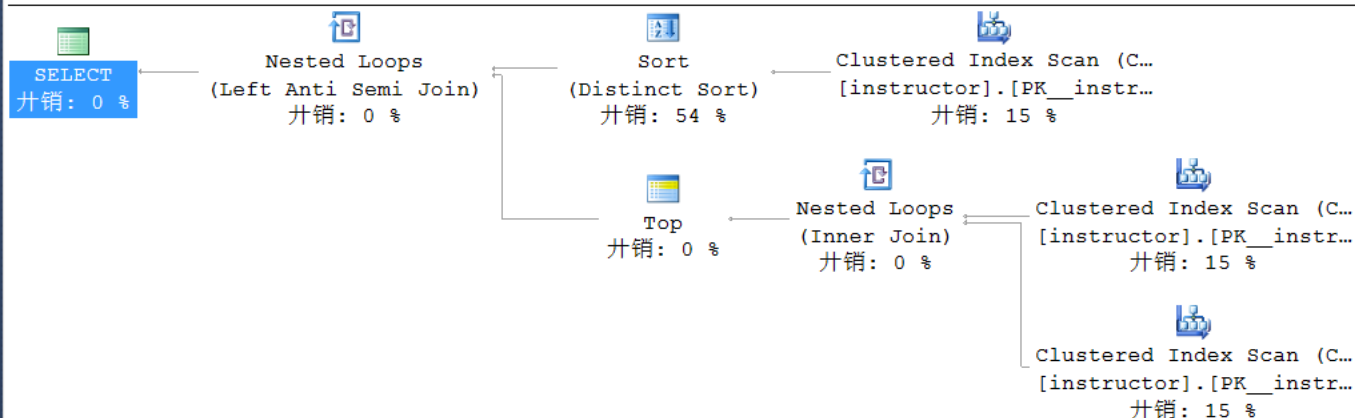
```
select max(salary) as maxsalary from instructor
```

150 %

消息 执行计划

查询 1: (与该批有关的) 查询开销: 87%

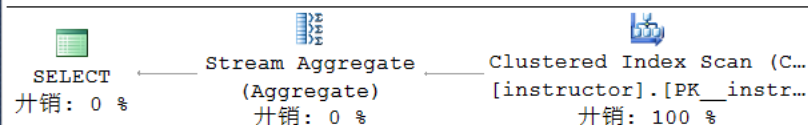
(select distinct salary?from instructor) except (select distinct T.salary?from instructor as T, instructor as S?where



自下而上执行查询树中的关系代数操作

查询 2: (与该批有关的) 查询开销: 13%

select max(salary) as maxsalary?from instructor



6.1 Relational Algebra

6.1.1 Fundamental Operations

- Six basic operators
 - select: σ
 - project: Π
 - union: $\cup$
 - set difference: $-$
 - Cartesian product: $\times$
 - rename: ρ
- The operators take one or two relations as inputs and produce a new relation as a result.

Select Operation

- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Defined as:

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

- SQL statement

Select *

From *r*

Where **P**

Where p is a formula in propositional calculus consisting of **terms** connected by : $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

Each **term** is one of:

$\langle \text{attribute} \rangle op \langle \text{attribute} \rangle$ or $\langle \text{constant} \rangle$

where op is one of: $=, \neq, >, \geq, <, \leq$

- Example of selection: $\sigma_{dept_name="Physics"}(instructor)$

■ Example of selection: $\sigma_{dept_name="Physics"}(instructor)$

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

Project Operation

- Notation:

$$\Pi_{A_1, A_2, \dots, A_k}(r)$$

- SQL statement

```
Select  $A_1, A_2, \dots, A_k$   
From  $r$ 
```

where A_1, A_2 are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets
- Example: To eliminate the *dept_name* attribute of *instructor*

$$\Pi_{ID, name, salary}(instructor) \quad \blacktriangleright$$

Composition Operations

- Several relational algebra operations can be composed together into a **relational algebra expression**
- E.g.2. $\Pi_{Id, name, salary}(\sigma_{dept_name = 'Physic'}(instructor))$ 

Union Operation

- Notation: $r \cup s$

- Defined as:

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

- For $r \cup s$ to be valid.

1. r, s must have the *same* **arity** (same number of attributes)
2. The attribute domains must be **compatible** (example: 2nd column of r deals with the same type of values as does the 2nd column of s)

SQL statement

$r \text{ **Union** } s$

$r \text{ **Union All** } s$

Union Operation

- Example: to find all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or in both

$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2009}(section)) \cup$$
$$\Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2010}(section))$$

Set Difference Operation

- Notation $r - s$

- Defined as:

$$r - s = \{t \mid t \in r \textbf{ and } t \notin s\}$$

SQL statement

r *Except* s

- Set differences must be taken between **compatible** relations.

- r and s must have the **same** arity
- attribute domains of r and s must be compatible

■

- Note: $r \cap s = r - (r - s)$

Set Difference Operation

- Example: to find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2009}(section)) - \Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2010}(section))$$

Cartesian-Product Operation

- Notation $r \times s$

- Defined as:

$$r \times s = \{t \mid t = t_1 t_2 \textbf{ and } t_1 \in r \textbf{ and } t_2 \in s \}$$

- Assume that attributes of $r(R)$ and $s(S)$ are disjoint. (That is, $R \cap S = \emptyset$).
- If attributes of $r(R)$ and $s(S)$ are not disjoint, then renaming must be used.

Cartesian-Product

■ E.g.1

$$t_1$$

A	B
α	1
β	2

r

$$t_2$$

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

$$t$$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

$r \times s$

Fig.6.05

■ SQL Statement:

Select *

From r, s

■ Cartesian-product operation on relation r and s

$$r \times s = \{t \mid t = t_1 t_2 \text{ and } t_1 \in r \text{ and } t_2 \in s\}$$

Cartesian-Product

- For relation $r(R)$ and $s(S)$, $r \times s$ is a relation whose schema is R' , which is the **concatenation** of R and S
 - $R(A_1, A_2, \dots, A_m), S(B_1, B_2, \dots, B_n)$
 $R'(A_1, A_2, \dots, A_m, B_1, B_2, \dots, B_n)$
 - for $t \in r \times s$, there are $t_1 \in r$ and $t_2 \in s$, such that
$$t[R] = t_1[R]$$
$$t[S] = t_2[S]$$
 - assume that attributes of $r(R)$ and $s(S)$ are **disjoint**. i.e. $\{A_1, A_2, \dots, A_m\} \cap \{B_1, B_2, \dots, B_n\} = \emptyset$
 - if attributes of $r(R)$ and $s(S)$ are not disjoint, then renaming must be used.

Fundamental Operations vs *Select* statement in SQL

- A typical SQL query has the form

select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

- A_i represents attributes
- r_i represents relations
- P is a predicate
- Equivalent to the relational algebra expression

$$\Pi_{A_1, A_2, \dots, A_n}(\sigma_P(r_1 \times r_2 \times \dots \times r_m))$$

Rename Operation

- The rename operation on relation r or relational algebra expression E

$$\rho_x(r), \rho_x(E)$$

- renaming r or E as X
- If a relational-algebra expression E has arity n , then

$$\rho_x(A_1, A_2, \dots, A_n)(E)$$

returns the result of expression E under the name X , and with the attributes renamed to $A_1, A_2, \dots, A_n$

- E.g. $\rho_s(A_1, A_2, \dots, A_n)(r(B_1, B_2, \dots, B_n)) = s(A_1, A_2, \dots, A_n)$

Rename Operation

– An Example

- Find the highest *salary* in the *univ*ersity
- Principle
 - /*将对一个关系 *instructor* 中同一个属性 *salary* 的不同取值的比较转换为利用 换名操作、迪卡尔乘积和选择操作对关系 *instructor* 和它的副本 $\rho_d(\textit{instructor})$ 在同一属性 *salary* 上取值的比较, refer to Fig.6.0.6
- Step1.
 $\textit{instructor} \times \rho_d(\textit{instructor})$
- Step2. all tuples that have *non-largest salary* values in *instructor* relation:
$$\sigma_{\textit{instructor.salary} < d.\textit{salary}}(\textit{instructor} \times \rho_d(\textit{instructor}))$$
 - the result are tuple t_2, t_3, t_6

Rename Operation

– An Example (cont.)

<i>ins.ID</i>	<i>ins.name</i>	<i>ins.salary</i>
120005	Zhang	1000
130007	Wang	2000
198032	Li	3000

(a) *instructor*

<i>d.ID</i>	<i>d.name</i>	<i>d.salary</i>
120005	Zhang	1000
130007	Wang	2000
198032	Li	3000

(b) $d = \rho_d(\textit{instructor})$

Fig.2.06-1 relation *instructor* and *d*

instructor

d

	<i>ins.ID</i>	<i>ins.name</i>	<i>ins.salary</i>	<i>d.ID</i>	<i>d.name</i>	<i>d.salary</i>
	120005	Zhang	1000	120005	Zhang	1000
t_2	120005	Zhang	1000	130007	Wang	2000
t_3	120005	Zhang	1000	198032	Li	3000
	130007	Wang	2000	120005	Zhang	1000
	130007	Wang	2000	130007	Wang	2000
t_6	130007	Wang	2000	198032	Li	3000
	198032	Li	3000	120005	Zhang	1000
	198032	Li	3000	130007	Wang	2000
	198032	Li	3000	198032	Li	3000

Fig.2.06-2 $instructor \times \rho_d(instructor)$

Rename Operation

– An Example (cont.)

- Step3. all non-largest salary values in relation *instructor*

$\Pi_{instructor.salary}(\$

$\sigma_{instructor.salary < d.salary} (instructor \times \rho_d (instructor))$

- the result is $\{1000, 2000\}$

- Step4. All largest balance values in relation *account*

$\Pi_{salary}(instructor) \text{ — }$

$\Pi_{instructor.salary}(\$

$\sigma_{instructor.salary < d.salary} (instructor \times \rho_d (instructor))$

- the result is $\{1000, 2000, 3000\} \text{ — } \{1000, 2000\} = \{3000\}$

Rename Operation

– An Example (cont.)

- This query can also be solved by *Select* statement in SQL, by means of
 - *tuple variables* in § 3.3.5?
 - *set comparison* in § 3.7.2?
 - *aggregate function* *max* in § 3.5?
- Homework and Questions
 - 是否可用下式代替step4中的查询表达式
$$\Pi_{instructor.salary}(\sigma_{instructor.salary \geq d.salary}(instructor \times \rho_d(instructor)))$$
 - 如何求解 “Find the *smallest salary* in the university ”

6.1.2 Formal Definition of Relational Algebra

- Relational algebra expression E is defined as follows
 - a basic expression is
 - a relation r in the database, or
 - a constant relation $C = \{c_1, c_2, \dots, c_n\}$, which is written by listing its tuples in the relation
 - a general expression is defined as:
 - if E_1 and E_2 are relational algebra expressions, then
 - $E_1 \cup E_2$
 - $E_1 - E_2$
 - $E_1 \times E_2$

Formal Definition of Relational Algebra (cont.)

- $\sigma_p (E_1)$
- $\Pi_s (E_1),$
- $\rho_x (E_1)$

are all relational algebra expressions

6.1.3 Additional Operations

- Additional operations
 - $\cap$, set-intersection (集合交)
 - ∞ , $\bowtie$ natural-join (自然、外连接)
 - $\leftarrow$, assignment (赋值)
- With respect to querying powers, additional operations are equivalent with fundamental operations, i.e. can be replaced (or re-represented) by the six basic operations operations
 - a additional operation can be replaced/re-represented by basic operations
 - additional operations simplify representations of queries

6.1.3.1 The *Set-Intersection* Operation

- For relation r and s , set-intersection operation on r and s is defined as

$$r \cap s = \{ t \mid t \in r \text{ and } t \in s \}$$

- r, s have the *same arity*
 - attributes of r and s are compatible
-
- $r \cap s = r - (r - s)$
-
- SQL statement
 - intersect

6.1.3.2 The Natural-join Operation

E.g.1

(1)

A	<i>B</i>	C	<i>D</i>
α	<i>1</i>	α	<i>a</i>
β	2	γ	a
γ	4	β	b
α	<i>1</i>	γ	<i>a</i>
δ	<i>2</i>	β	<i>b</i>

r

(1)

<i>B</i>	<i>D</i>	E
<i>1</i>	<i>a</i>	α
3	a	β
<i>1</i>	<i>a</i>	γ
<i>2</i>	<i>b</i>	δ
3	b	ϵ

s

(1)

A	B	C	D	E
α	<i>1</i>	α	<i>a</i>	α
α	<i>1</i>	α	<i>a</i>	γ
α	<i>1</i>	γ	a	α
α	<i>1</i>	γ	a	γ
δ	2	β	b	δ

r $\bowtie$ *s*

Fig.6.07

Natural-join (cont.)

- For relations $r(R)$ and $s(S)$, assuming $R \cap S = \{A_1, A_2, \dots, A_n\}$, the natural join of r and s , denoted by $r \bowtie s$, is a relation on schema $R \cup S$,

$$r \bowtie s =$$

$$\Pi_{R \cup S} (\sigma_{r.A_1=s.A_1 \wedge r.A_2=s.A_2 \wedge \dots \wedge r.A_n=s.A_n} (r \times s))$$

- E.g.1 (cont.)

$$R = (A, B, C, D), S = (B, D, E), r(R) \text{ and } s(S),$$

$$r \bowtie s =$$

$$\Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B=s.B \wedge r.D=s.D} (r \times s))$$

- defined on the schema (A, B, C, D, E)



2.3.2 The Natural-join Operation (cont.)

- Natural-join operation $\bowtie$ combine σ and $\times$ on some relations into one operation
 - to improve execution efficiency of query 
 - for more details, refer to Query Optimization in Chapter 13

Natural-join

- E.g.2 Find the *names* of all instructors in the Comp.Sci department together with the course titles of all courses the instructors teach

<i>name</i>	<i>title</i>
Brandt	Game Design
Brandt	Image Processing
Katz	Image Processing
Katz	Intro. to Computer Science
Srinivasan	Intro. to Computer Science
Srinivasan	Robotics
Srinivasan	Database System Concepts

Figure 6.16 Result of

$\Pi_{name, title} (\sigma_{dept_name = \text{"Comp. Sci."}} (instructor \bowtie teaches \bowtie course))$.

Natural-join

- Properties of natural join

- $r \bowtie (s \bowtie t) = (r \bowtie s) \bowtie t$
- if $R \cap S = \Phi$, then $r(R) \bowtie s(S) = r(R) \times s(S)$

Select *customer-name, loan_number, amounts*
From *borrower natural_join loan*

6.1.3.3 The Assignment Operation

- **Assign** ($\leftarrow$) *relation algebra expression* E to a *relational variable* var_r , i.e. denoting E as var_r .

$$\text{var_r} \leftarrow E$$

- The assignment operation ($\leftarrow$) provides a convenient way to express complex queries
- E.g. Write $r \div s$ as

$$\text{temp1} \leftarrow \Pi_{R-S}(r)$$

$$\text{temp2} \leftarrow \Pi_{R-S}((\text{temp1} \times s) - \Pi_{R-S,S}(r))$$

$$\text{result} = \text{temp1} - \text{temp2}$$

Assignment (cont.)

- Assignment must always be made to a *temporary* relation variable
 - assignments to *permanent* relations constitute a database modification

6.1.3.4 Outer join

- The outer join operation is an extension of the join operation to deal with missing information

ID	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
33456	Gold	Physics	87000	null	null	null	null
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
58583	Califieri	History	62000	null	null	null	null
76543	Singh	Finance	80000	null	null	null	null
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

Figure 6.17 Result of *instructor* ⋈ *teaches*.

Outer join

ID	course_id	sec_id	semester	year	name	dept_name	salary
10101	CS-101	1	Fall	2009	Srinivasan	Comp. Sci.	65000
10101	CS-315	1	Spring	2010	Srinivasan	Comp. Sci.	65000
10101	CS-347	1	Fall	2009	Srinivasan	Comp. Sci.	65000
12121	FIN-201	1	Spring	2010	Wu	Finance	90000
15151	MU-199	1	Spring	2010	Mozart	Music	40000
22222	PHY-101	1	Fall	2009	Einstein	Physics	95000
32343	HIS-351	1	Spring	2010	El Said	History	60000
33456	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Gold	Physics	87000
45565	CS-101	1	Spring	2010	Katz	Comp. Sci.	75000
45565	CS-319	1	Spring	2010	Katz	Comp. Sci.	75000
58583	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Califieri	History	62000
76543	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Singh	Finance	80000
76766	BIO-101	1	Summer	2009	Crick	Biology	72000
76766	BIO-301	1	Summer	2010	Crick	Biology	72000
83821	CS-190	1	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-190	2	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-319	2	Spring	2010	Brandt	Comp. Sci.	92000
98345	EE-181	1	Spring	2009	Kim	Elec. Eng.	80000

Figure 6.18 Result of *teaches* $\bowtie$ *instructor*.

§ 6.1.4 Extended Relational Algebra Operations

Generalized project, aggregate functions

- these operations extends query powers of basic and additive relational algebra operations

6.1.4.1 Generalized Project (一般化/扩展投影操作)

- Generalized project extends the *projection* operation by allowing arithmetic functions to be used in the projection list.

$$\Pi_{F_1, F_2, \dots, F_n}(E)$$

, where

- E is any relational-algebra expression
- each of $F_1, F_2, \dots, F_n$ are arithmetic expressions involving constants and attributes in the schema of E .

2.4.1 Generalized Project (cont.)

- involving arithmetic operation on values of *different attributes in one relation*,

e.g. $E.A_i - E.A_j$

- E.g.

$\Pi_{ID, name, department_name, salary/12} (instructor)$

6.1.4.2 Aggregation Function

- Aggregation functions
 - mapping of a collection of attribute values in a relation r into a **single** value, i.e. attribute domains $\rightarrow$ domain
 - **avg**: average value
 - **min**: minimum value
 - **max**: maximum value
 - **sum**: sum of values
 - **count**: number of values
- A *convenient* scheme for computing and comparing of the values in one attribute of the relation, *in contrast with* 

Aggregate Functions in SQL

- *group by* subclause and *aggregate* function

```
Select  {  $A_1, A_2, \dots, A_i$ , } ag_fun( $A_{i+1}$ ), ..., ag_fun( $A_{i+k}$ )  
From     $r_1, r_2, \dots, r_m$   
Where    $P1$   
{ Group by  $A_1, A_2, \dots, A_i$   
  { having  $P2$  } }
```

- notes:

- $P1$ is defined on all attributes of $r_1, r_2, \dots, r_m$
- $P2$ is defined on $A_1, A_2, \dots, A_i, \dots, A_j, \dots, A_k, \dots, A_n$, as the constraints on the group formed by **Group by** A_1, A_2, A_j

Aggregation Function (cont.)

- **Aggregate operation** in relational algebra

$$G_1, G_2, \dots, G_n \mathcal{G} F_1(A_1), F_2(A_2), \dots, F_m(A_m) (E)$$

- E is any relational-algebra expression
- $G_1, G_2, \dots, G_n$ is a list of attributes on which to group (may be empty)
 - a list of attributes in E on which to group (分组), i.e. tuples in E are classified into several groups $E_1, E_2, \dots, E_n$, each group E_i is a subset of E , and tuples in each group E_i have the same values for $G_1, G_2, \dots, G_n$
- if $G_1, G_2, \dots, G_n$ do not appear, aggregations are conducted on all tuples in E , that is, E are not classified into groups

§ 2.2.2 Aggregation Function (cont.)

- each A_i is an attribute on which to conduct aggregation functions F_i
 - also the attribute names for aggregation results
- aggregation functions F_i , e.g. **avg, count, min, max, sum**
- computing procedure
 - separating E into groups on the basis of $G_1, G_2, \dots, G_n$
 - computing $F_1, F_2, \dots, F_n$ on each group, and return computing results denoted as $A_1, A_2, \dots, A_n$

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

Figure 6.19 Tuples of the *instructor* relation, grouped by the *dept_name* attribute

<i>dept_name</i>	<i>salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Figure 6.20 The result relation for the query “Find the average salary in each department”.

From a comparison of the relational algebra operations and the SQL operations, it should be clear that there is a close connection between the two. A typical SQL query has the form:

```
select  $A_1, A_2, \dots, A_n$ 
from  $r_1, r_2, \dots, r_m$ 
where  $P$ 
```

Each A_i represents an attribute, and each r_i a relation. P is a predicate. The query is equivalent to the multiset relational-algebra expression:

$$\Pi_{A_1, A_2, \dots, A_n}(\sigma_P(r_1 \times r_2 \times \dots \times r_m))$$

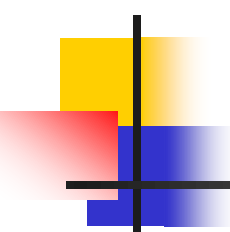
If the **where** clause is omitted, the predicate P is **true**.

More complex SQL queries can also be rewritten in relational algebra. For example, the query:

```
select  $A_1, A_2, \text{sum}(A_3)$ 
from  $r_1, r_2, \dots, r_m$ 
where  $P$ 
group by  $A_1, A_2$ 
```

is equivalent to:

$$A_1, A_2 \mathcal{G}_{\text{sum}(A_3)}(\Pi_{A_1, A_2, \dots, A_n}(\sigma_P(r_1 \times r_2 \times \dots \times r_m)))$$



Have a rest
